คู่มือ API ระบบยืม-คืนบอร์ดเกม (Boardgame Service)

รหัสนิสิต: 67160197 | Base URL: `http://localhost:3000`

เอกสารนี้อธิบาย endpoint ทั้งหมดของระบบ พร้อมรูปแบบ request, response, รหัส error และตัวอย่างการเรียกใช้งาน

สารบัญ

1. ภาพรวมและการเรียกใช้งาน
2. รูปแบบข้อมูลตอบกลับ
3. รายการรหัส error
4. สรุป endpoint ทั้งหมด
5. Health check, เอกสาร และหน้าเว็บ
6. Boardgames: บอร์ดเกม
7. Members: สมาชิก
8. Borrowings: การยืม-คืน
9. External: การแจ้งเตือนภายนอก
10. ลำดับการทดสอบแบบง่าย

---

1. ภาพรวมและการเรียกใช้งาน

1.1 ข้อกำหนดทั่วไป

หัวข้อ	รายละเอียด
Base path	`/api/v1` (ยกเว้น `/` และ `/api-docs`)
รูปแบบข้อมูล	ส่งและรับเป็น JSON (`Content-Type: application/json`)
ขนาด body สูงสุด	10 KB (เกินกว่านี้ได้ `413`)
CORS	อนุญาต origin ที่ตั้งไว้ใน `ALLOWED_ORIGIN` (ค่าเริ่มต้นคือ Live Server `127.0.0.1:5500` และ `localhost:5500`)
Methods ที่ใช้ได้	`GET`, `POST`, `PUT`, `PATCH`, `DELETE`
วันที่	รูปแบบ `YYYY-MM-DD` เช่น `2026-10-31`
id	จำนวนเต็มบวก เช่น `1`, `25` (ถ้าส่งค่าอื่นจะได้ `400 INVALID_ID`)

1.2 วิธีเรียกใช้งาน

ระบบต้องทำงานอยู่ก่อน (ดูขั้นตอนใน README) จากนั้นเรียกผ่านเครื่องมือใดก็ได้ เช่น Postman, Swagger UI (`/api-docs`) หรือ curl

ตัวอย่างในคู่มือนี้ใช้ **Windows CMD / PowerShell** ด้วย `curl.exe` ตามที่ README แนะนำ

- ใน CMD การส่ง JSON ต้อง escape เครื่องหมาย `"` ภายใน body ด้วย `\\\"` ตามตัวอย่าง
- ถ้าใช้ Linux / macOS / Git Bash ให้เปลี่ยนเป็น `curl` และครอบ JSON ด้วยเครื่องหมาย `'...'` ได้ เช่น `-d '{"memberId":3,"boardgameId":1}'`

ตัวอย่าง request ที่ใช้ได้จริงใน CMD:

```
curl.exe -X POST http://localhost:3000/api/v1/borrowings -H "Content-Type: application/json" -d "{\\\"memberId\\\":3,\\\"boardgameId\\\":1}"
```

---

2. รูปแบบข้อมูลตอบกลับ

2.1 เมื่อสำเร็จ

```json
{
  "message": "สำเร็จ",
  "data": {}
}
```

- `message` ข้อความภาษาไทยที่บอกผลลัพธ์
- `data` ข้อมูลที่ต้องการ (เป็น object สำหรับข้อมูลเดี่ยว หรือ array สำหรับรายการ)

2.2 เมื่อเกิด error

```json
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "กรุณาระบุ name, minPlayers, maxPlayers ให้ครบถ้วน"
  }
}
```

- `code` รหัสสำหรับโปรแกรมตรวจสอบ (ดูหัวข้อ 3)
- `message` คำอธิบายสำหรับมนุษย์ (ภาษาไทย)

---

3. รายการรหัส error

HTTP Status	code	ความหมาย	ตัวอย่างสาเหตุ
400	`VALIDATION_ERROR`	ข้อมูลไม่ถูกต้องหรือไม่ครบ	ไม่ส่งชื่อ, วันที่ผิดรูปแบบ, ค่าใน query ไม่ถูกต้อง
400	`INVALID_ID`	id ใน URL ไม่ใช่จำนวนเต็มบวก	`/api/v1/members/abc`
400	`entity.parse.failed`	JSON ใน body ผิดรูปแบบ	ลืมปิดเครื่องหมาย `}`
404	`NOT_FOUND`	ไม่พบข้อมูลที่ต้องการ	id ไม่มีในระบบ
404	`ROUTE_NOT_FOUND`	ไม่มี path นี้ในระบบ	`/api/v1/xxx`
409	`DUPLICATE_NAME`	ชื่อบอร์ดเกมซ้ำ	เพิ่ม "Catan" ซ้ำ
409	`DUPLICATE_EMAIL`	อีเมลสมาชิกซ้ำ	เพิ่มอีเมลที่มีอยู่แล้ว
409	`GAME_NOT_AVAILABLE`	บอร์ดเกมกำลังถูกยืมอยู่	ยืมเกมที่ยังไม่ได้คืน
409	`ALREADY_RETURNED`	รายการนี้ถูกคืนไปแล้ว	คืนซ้ำ
413	`entity.too.large`	body ใหญ่เกิน 10 KB	ส่งข้อมูลขนาดผิดปกติ
503	`EXTERNAL_SERVICE_UNAVAILABLE`	ส่งการแจ้งเตือนภายนอกไม่สำเร็จ	ntfy.sh ใช้งานไม่ได้
500	`INTERNAL_SERVER_ERROR`	ข้อผิดพลาดภายในระบบ (ดูรายละเอียดใน log ของ server)	ฐานข้อมูลไม่พร้อม

4. สรุป endpoint ทั้งหมด

#	Method	Path	หน้าที่	Success
0	GET	`/`	ตรวจสอบว่า server ทำงาน	200
1	GET	`/api/v1/boardgames`	รายการบอร์ดเกม (กรองด้วย `?players=`)	200
2	GET	`/api/v1/boardgames/:id`	ดูบอร์ดเกม 1 เกม	200
3	POST	`/api/v1/boardgames`	เพิ่มบอร์ดเกม	201
4	PUT	`/api/v1/boardgames/:id`	แก้ไขบอร์ดเกม	200
5	DELETE	`/api/v1/boardgames/:id`	ลบบอร์ดเกม	200
6	GET	`/api/v1/members`	รายชื่อสมาชิก	200
7	GET	`/api/v1/members/:id`	ดูสมาชิก 1 คน	200
8	GET	`/api/v1/members/:id/borrowings`	ประวัติการยืมของสมาชิก	200
9	POST	`/api/v1/members`	เพิ่มสมาชิก	201
10	PUT	`/api/v1/members/:id`	แก้ไขสมาชิก	200
11	DELETE	`/api/v1/members/:id`	ลบสมาชิก	200
12	GET	`/api/v1/borrowings`	รายการยืม-คืน (กรองได้)	200
13	GET	`/api/v1/borrowings/:id`	ดูรายการยืม 1 รายการ	200
14	POST	`/api/v1/borrowings`	ยืมบอร์ดเกม	201
15	PATCH	`/api/v1/borrowings/:id/return`	คืนบอร์ดเกม	200
16	GET	`/api/v1/external/notify?message=...`	ส่งการแจ้งเตือนด้วยตนเอง	200

นอกจากนี้มีเส้นทางสำหรับเอกสารและหน้าเว็บ ได้แก่ `/api-docs` (Swagger UI) และ `/app` (หน้าเว็บ)

---

5. Health check, เอกสาร และหน้าเว็บ

5.1 GET `/`

ใช้ตรวจสอบว่า server ทำงานอยู่หรือไม่

Response 200

```json
{ "message": "Boardgames API พร้อมใช้งานแล้ว" }
```

```
curl.exe http://localhost:3000/
```

5.2 GET `/api-docs`

เปิดหน้า Swagger UI เพื่อดูเอกสาร API (อ่านจากไฟล์ `openapi.yaml`)

5.3 GET `/app`

เปิดหน้าเว็บจัดการระบบ ใช้ API ตัวเดียวกับในคู่มือนี้

---

6. Boardgames: บอร์ดเกม

ข้อมูลบอร์ดเกมมีฟิลด์ดังนี้

ฟิลด์	ชนิด	คำอธิบาย
`id`	number	รหัสบอร์ดเกม
`name`	string	ชื่อเกม (ไม่เกิน 150 ตัวอักษร ห้ามซ้ำ)
`minPlayers`	number	จำนวนผู้เล่นขั้นต่ำ (ตั้งแต่ 1)
`maxPlayers`	number	จำนวนผู้เล่นสูงสุด (ต้องไม่น้อยกว่า minPlayers)
`createdAt`	string	วันเวลาที่เพิ่มเข้าระบบ (มีเฉพาะตอน GET)

6.1 GET `/api/v1/boardgames` : ดูรายการบอร์ดเกม

ใช้ query parameter `players` (ไม่บังคับ) เพื่อค้นหาเกมที่รองรับจำนวนผู้เล่นนั้น (`minPlayers ≤ players ≤ maxPlayers`)

Query	ชนิด	บังคับ	คำอธิบาย
`players`	จำนวนเต็มบวก	ไม่	จำนวนผู้เล่นที่ต้องการค้นหา

เรียงตาม id จากน้อยไปมาก

Response 200

```json
```

```
{
  "message": "สำเร็จ",
  "data": [
    { "id": 1, "name": "Catan", "minPlayers": 3, "maxPlayers": 4, "createdAt":
    { "id": 2, "name": "Codenames", "minPlayers": 4, "maxPlayers": 8, "createdA
    { "id": 3, "name": "Uno", "minPlayers": 2, "maxPlayers": 10, "createdAt": "
  ]
}
```


Error: `400 VALIDATION_ERROR` (`players ต้องเป็นจำนวนเต็มบวก`)

```
curl.exe http://localhost:3000/api/v1/boardgames
curl.exe "http://localhost:3000/api/v1/boardgames?players=4"
```

6.2 GET `/api/v1/boardgames/:id` : ดูบอร์ดเกม 1 เกม

Response 200

json


```
{
  "message": "สำเร็จ",
  "data": { "id": 1, "name": "Catan", "minPlayers": 3, "maxPlayers": 4, "create
}
```


Error: `400 INVALID_ID`, `404 NOT_FOUND` (`ไม่พบข้อมูลของบอร์ดเกม`)

```
curl.exe http://localhost:3000/api/v1/boardgames/1
```

6.3 POST `/api/v1/boardgames` : เพิ่มบอร์ดเกม

Request body

ฟิลด์	บังคับ	กฎ
`name`	ใช่	ไม่ว่าง ไม่เกิน 150 ตัวอักษร (ตัดช่องว่างหัวท้ายก่อนนับ)
`minPlayers`	ใช่	จำนวนเต็ม ตั้งแต่ 1 และไม่มากกว่า `maxPlayers`
`maxPlayers`	ใช่	จำนวนเต็ม ไม่น้อยกว่า `minPlayers`

json

```
{ "name": "Ticket to Ride", "minPlayers": 2, "maxPlayers": 5 }
```

Response 201

```
json
{
  "message": "เพิ่มข้อมูลสำเร็จ",
  "data": { "id": 4, "name": "Ticket to Ride", "minPlayers": 2, "maxPlayers": 5
}
```

Error: `400 VALIDATION_ERROR` (ข้อมูลไม่ครบหรือไม่ถูกต้อง), `409 DUPLICATE_NAME` (`ชื่อบอร์ดเกมนี้มีอยู่ในระบบแล้ว`)

```
curl.exe -X POST http://localhost:3000/api/v1/boardgames -H "Content-Type:
application/json" -d "{\"name\":\"Ticket to
Ride\",\"minPlayers\":2,\"maxPlayers\":5}"
```

6.4 PUT `/api/v1/boardgames/:id` : แก้ไขบอร์ดเกม

ต้องส่งข้อมูลครบทั้ง 3 ฟิลด์ (เป็นการแทนที่ทั้งรายการ) กฎเหมือนกับการเพิ่ม

Response 200

```
json
{
  "message": "แก้ไขข้อมูลสำเร็จ",
  "data": { "id": 4, "name": "Ticket to Ride Europe", "minPlayers": 2, "maxPlay
}
```

Error: `400 INVALID_ID`, `400 VALIDATION_ERROR`, `404 NOT_FOUND` (`ไม่พบข้อมูลบอร์ดเกม`), `409 DUPLICATE_NAME`

```
curl.exe -X PUT http://localhost:3000/api/v1/boardgames/4 -H "Content-Type:
application/json" -d "{\"name\":\"Ticket to Ride
Europe\",\"minPlayers\":2,\"maxPlayers\":5}"
```

6.5 DELETE `/api/v1/boardgames/:id` : ลบบอร์ดเกม

ประวัติการยืมของเกมนี้จะถูกลบไปด้วยอัตโนมัติ

Response 200

```
json
```

```
{ "message": "ลบข้อมูลสำเร็จ" }
```

Error: `400 INVALID_ID`, `404 NOT_FOUND`

```
curl.exe -X DELETE http://localhost:3000/api/v1/boardgames/4
```

---

7. Members: สมาชิก

ข้อมูลสมาชิกมีฟิลด์ `id`, `name`, `email`, `phone`

7.1 GET `/api/v1/members` : รายชื่อสมาชิกทั้งหมด

เรียงตาม id จากน้อยไปมาก

Response 200

json

```
{
  "message": "สำเร็จ",
  "data": [
    { "id": 1, "name": "สมชาย ใจดี", "email": "somchai@example.com", "phone": "08
  ]
}
```

```
curl.exe http://localhost:3000/api/v1/members
```

7.2 GET `/api/v1/members/:id` : ดูสมาชิก 1 คน

Response 200

json

```
{
  "message": "สำเร็จ",
  "data": { "id": 1, "name": "สมชาย ใจดี", "email": "somchai@example.com", "phone
}
```

Error: `400 INVALID_ID`, `404 NOT_FOUND` (`ไม่พบข้อมูลสมาชิก`)

```
curl.exe http://localhost:3000/api/v1/members/1
```

7.3 GET `/api/v1/members/:id/borrowings` : ประวัติการยืมของสมาชิก

เรียงจากการยืมล่าสุดไปเก่าสุด

Response 200

```json
{
  "message": "สำเร็จ",
  "data": [
    {
      "id": 2,
      "memberId": 1,
      "memberName": "สมชาย ใจดี",
      "boardgameId": 2,
      "boardgameName": "Codenames",
      "borrowDate": "2026-08-20",
      "returnDate": null,
      "status": "BORROWED"
    }
  ]
}
```

Error: `400 INVALID_ID`, `404 NOT_FOUND` (`ไม่พบข้อมูลสมาชิก`)

```
curl.exe http://localhost:3000/api/v1/members/1/borrowings
```

7.4 POST `/api/v1/members` : เพิ่มสมาชิก

Request body

ฟิลด์	บังคับ	กฎ
`name`	ใช่	ไม่ว่าง ไม่เกิน 100 ตัวอักษร
`email`	ใช่	รูปแบบอีเมลถูกต้อง (เช่น `xxx@xxx.xxx`) ไม่เกิน 100 ตัวอักษร ห้ามซ้ำ
`phone`	ใช่	ไม่ว่าง ไม่เกิน 20 ตัวอักษร

```json
{ "name": "ปรีชา ชอบเล่น", "email": "preecha@example.com", "phone": "0855556666" }
```

Response 201

```json
{
  "message": "เพิ่มข้อมูลสำเร็จ",
  "data": { "id": 4, "name": "ปรีชา ชอบเล่น", "email": "preecha@example.com", "pho
}
```

Error: `400 VALIDATION_ERROR`, `409 DUPLICATE_EMAIL` (`อีเมลนี้ถูกใช้งานในระบบแล้ว`)

```
curl.exe -X POST http://localhost:3000/api/v1/members -H "Content-Type:
application/json" -d "{\"name\":\"ปรีชา ชอบ
เล่น\",\"email\":\"preecha@example.com\",\"phone\":\"0855556666\
```

7.5 PUT `/api/v1/members/:id` : แก้ไขสมาชิก

ต้องส่งข้อมูลครบทั้ง 3 ฟิลด์ กฎเหมือนกับการเพิ่ม

Response 200

```json
{
  "message": "แก้ไขข้อมูลสำเร็จ",
  "data": { "id": 4, "name": "ปรีชา ชอบเล่น", "email": "preecha.new@example.com",
}
```

Error: `400 INVALID_ID`, `400 VALIDATION_ERROR`, `404 NOT_FOUND` (`ไม่พบข้อมูลสมาชิก`), `409 DUPLICATE_EMAIL`

```
curl.exe -X PUT http://localhost:3000/api/v1/members/4 -H "Content-Type:
application/json" -d "{\"name\":\"ปรีชา ชอบ
เล่น\",\"email\":\"preecha.new@example.com\",\"phone\":\"08555566
```

7.6 DELETE `/api/v1/members/:id` : ลบสมาชิก

ประวัติการยืมของสมาชิกคนนี้จะถูกลบไปด้วยอัตโนมัติ

Response 200

```json
{ "message": "ลบข้อมูลสำเร็จ" }
```

Error: `400 INVALID_ID`, `404 NOT_FOUND`

```
curl.exe -X DELETE http://localhost:3000/api/v1/members/4
```

---

8. Borrowings: การยืม-คืน

ข้อมูลการยืมมีฟิลด์ดังนี้

ฟิลด์	ชนิด	คำอธิบาย
`id`	number	รหัสรายการยืม
`memberId`	number	รหัสสมาชิก
`memberName`	string	ชื่อสมาชิก
`boardgameId`	number	รหัสบอร์ดเกม
`boardgameName`	string	ชื่อบอร์ดเกม
`borrowDate`	string	วันที่ยืม (`YYYY-MM-DD`)
`returnDate`	string หรือ null	วันที่คืน (null = ยังไม่คืน)
`status`	string	`BORROWED` (กำลังยืม) หรือ `RETURNED` (คืนแล้ว)

กฎสำคัญ: บอร์ดเกม 1 เกมจะมีรายการที่สถานะ `BORROWED` ได้ไม่เกิน 1 รายการในเวลาเดียวกัน

8.1 GET `/api/v1/borrowings` : ดูรายการยืม-คืน

ทุก query parameter เป็นทางเลือก ใส่ซ้อนกันได้ (ใช้เงื่อนไข AND)

Query	ชนิด	คำอธิบาย
`memberId`	จำนวนเต็มบวก	กรองตามสมาชิก
`boardgameId`	จำนวนเต็มบวก	กรองตามบอร์ดเกม
`status`	`BORROWED` หรือ `RETURNED`	กรองตามสถานะ
`from`	`YYYY-MM-DD`	วันที่ยืมตั้งแต่วันนี้เป็นต้นไป
`to`	`YYYY-MM-DD`	วันที่ยืมถึงวันนี้

เรียงจากวันที่ยืมล่าสุดไปเก่าสุด

Response 200 เป็น array ของรายการยืมตามรูปแบบด้านบน

Error: `400 VALIDATION_ERROR` (เช่น `status=ABC`, `from=2026-13-01`, `memberId=abc`)

```
curl.exe http://localhost:3000/api/v1/borrowings
curl.exe "http://localhost:3000/api/v1/borrowings?status=BORROWED"
curl.exe "http://localhost:3000/api/v1/borrowings?memberId=1&from=2026-08-
01&to=2026-08-31"
```

8.2 GET `/api/v1/borrowings/:id` : ดูรายการยืม 1 รายการ

Response 200 ได้รายการยืม 1 รายการตามรูปแบบด้านบน

Error: `400 INVALID_ID`, `404 NOT_FOUND` (`ไม่พบรายการยืม`)

```
curl.exe http://localhost:3000/api/v1/borrowings/2
```

8.3 POST `/api/v1/borrowings` : ยืมบอร์ดเกม

ระบบจะตรวจว่าเกมยังว่างอยู่ก่อนบันทึก ขั้นตอนนี้ทำใน transaction และล็อกแถวของเกม จึงกันการยืมซ้อนกันได้แม้มีหลายคนกดพร้อมกัน

Request body

ฟิลด์	บังคับ	กฎ
`memberId`	ใช่	จำนวนเต็มบวก และต้องเป็นสมาชิกที่มีอยู่
`boardgameId`	ใช่	จำนวนเต็มบวก และต้องเป็นบอร์ดเกมที่มีอยู่
`borrowDate`	ไม่	รูปแบบ `YYYY-MM-DD` ถ้าไม่ส่งจะใช้วันที่วันนี้

json

```
{ "memberId": 3, "boardgameId": 1 }
```

Response 201

json

```
{
  "message": "บันทึกการยืมสำเร็จ",
  "data": {
    "id": 4,
    "memberId": 3,
    "boardgameId": 1,
    "borrowDate": "2026-10-31",
    "returnDate": null,
    "status": "BORROWED"
  }
}
```

เมื่อบันทึกสำเร็จ ระบบจะส่งการแจ้งเตือนผ่าน ntfy.sh ในเบื้องหลัง โดยไม่รอผลและไม่กระทบการตอบกลับ

Error

- `400 VALIDATION_ERROR` : `memberId` หรือ `boardgameId` ไม่ถูกต้อง หรือ `borrowDate` ผิดรูปแบบ
- `404 NOT_FOUND` : ไม่พบบอร์ดเกม (`ไม่พบข้อมูลบอร์ดเกม`) หรือไม่พบสมาชิก (`ไม่พบข้อมูลสมาชิก`)
- `409 GAME_NOT_AVAILABLE` : บอร์ดเกมกำลังถูกยืมอยู่ (`บอร์ดเกมนี้ถูกยืมอยู่ในขณะนี้`)

```
curl.exe -X POST http://localhost:3000/api/v1/borrowings -H "Content-Type: application/json" -d "{\\\"memberId\\\":3,\\\"boardgameId\\\":1}"
```

ยืมเกมเดิมซ้ำขณะที่ยังไม่ได้คืนจะได้ `409 GAME_NOT_AVAILABLE`

8.4 PATCH `/api/v1/borrowings/:id/return` : คืนบอร์ดเกม

เปลี่ยนสถานะจาก `BORROWED` เป็น `RETURNED` และบันทึกวันที่คืน

Request body (ไม่บังคับ)

ฟิลด์	บังคับ	กฎ
`returnDate`	ไม่	รูปแบบ `YYYY-MM-DD` และต้องไม่ก่อน `borrowDate` ถ้าไม่ส่งจะใช้วันที่วันนี้

```
json
{ "returnDate": "2026-10-31" }
```

ส่ง body เป็น `{}` ก็ได้ ระบบจะใช้วันที่วันนี้

Response 200

```
json
```

```
{
  "message": "คืนบอร์ดเกมสำเร็จ",
  "data": {
    "id": 4,
    "memberId": 3,
    "memberName": "มานะ พากเพียร",
    "boardgameId": 1,
    "boardgameName": "Catan",
    "borrowDate": "2026-10-31",
    "returnDate": "2026-10-31",
    "status": "RETURNED"
  }
}
```


Error

- `400 INVALID_ID`
- `400 VALIDATION_ERROR` : `returnDate` ผิดรูปแบบ หรือก่อนวันที่ยืม (`returnDate ต้องไม่ก่อน borrowDate`)
- `404 NOT_FOUND` : ไม่พบรายการยืม
- `409 ALREADY_RETURNED` : รายการนี้ถูกคืนไปแล้ว

```
curl.exe -X PATCH http://localhost:3000/api/v1/borrowings/4/return -H
"Content-Type: application/json" -d "{}"
```

---

9. External: การแจ้งเตือนภายนอก

9.1 GET `/api/v1/external/notify` : ส่งข้อความแจ้งเตือนด้วยตนเอง

ส่งข้อความไปยัง ntfy.sh (ตั้งค่าได้ใน `.env` ด้วย `NOTIFY_URL` และ `NOTIFY_TOPIC`) ระบบมีการ retry และ timeout อัตโนมัติตามค่า `EXTERNAL_*` ใน `.env`

Query	บังคับ	กฎ
`message`	ใช่	ไม่ว่าง ไม่เกิน 500 ตัวอักษร

Response 200

```
json
```

```
{
  "message": "ส่งการแจ้งเตือนสำเร็จ",
  "data": { "delivered": true, "attempts": 1 }
}
```

Error

- `400 VALIDATION_ERROR`: ไม่ได้ส่ง `message`, ว่าง, หรือยาวเกิน 500 ตัวอักษร
- `503 EXTERNAL_SERVICE_UNAVAILABLE`: ส่งไม่สำเร็จหลัง retry ครบ ข้อความที่ได้คือ `ทำรายการสำเร็จ แต่ระบบแจ้งเตือนภายนอกไม่พร้อมใช้งานในขณะนี้ (Fallback Executed)`

```
curl.exe "http://localhost:3000/api/v1/external/notify?message=ทดสอบการแจ้งเตือน"
```

> หมายเหตุ: ถ้าตั้ง `EXTERNAL_TIMEOUT_MS=1` ใน `.env` การเรียกนี้มักได้ `503` ซึ่งเป็นพฤติกรรมปกติของการทดสอบ retry (ดู README หัวข้อ 9) ค่าปกติคือ `2000`

---

10. ลำดับการทดสอบแบบง่าย

ทำตามลำดับนี้เพื่อตรวจว่าระบบทำงานครบทุกส่วน

1. `GET /` ควรได้ข้อความ "พร้อมใช้งานแล้ว"
2. `GET /api/v1/boardgames?players=4` ควรเห็นเกมที่รองรับ 4 คน
3. `POST /api/v1/members` เพิ่มสมาชิกใหม่ แล้วจดรหัสที่ได้
4. `POST /api/v1/borrowings` ยืมเกม id 1 ให้สมาชิกคนนั้น ควรได้ `201`
5. `POST /api/v1/borrowings` ยืมเกมเดิมซ้ำ ควรได้ `409 GAME_NOT_AVAILABLE`
6. `PATCH /api/v1/borrowings/:id/return` คืนเกม ควรได้ `status: RETURNED`
7. `GET /api/v1/members/:id/borrowings` ตรวจว่ามีประวัติการยืมครบ
8. `DELETE /api/v1/members/:id` ลบสมาชิกทดสอบ ประวัติการยืมจะถูกลบตามไปด้วย

สามารถ import `postman_collection.json` ใน Postman เพื่อทดสอบทุก endpoint พร้อมกันได้ (ควรตั้ง `EXTERNAL_TIMEOUT_MS=2000` ก่อน)